

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №3
на тему
**«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ.
КРИПТОСИСТЕМА РАБИНА»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Зашифрование и расшифрование текста	6
2.2	Работа с файлами	6
2.3	Обнаружение искажений	7
2.4	Задание ключей	8
	Заключение	9
	Приложение А (обязательное) Листинг программного кода	10
	Приложение Б (обязательное) Блок-схема алгоритма	16

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения асимметричных криптосистем на примере криптосистемы Рабина, стойкость которой основана на сложности извлечения квадратного корня по модулю составного числа. Задачи включают изучение математических основ криптосистемы, проектирование и реализацию программного средства, осуществляющего генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом, а также тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая зашифрование и расшифрование данных криптосистемой Рабина как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Реализация криптосистемы вынесена в отдельный модуль `encryption::rabin`, взаимодействие с пользователем осуществляется через текстовое меню, позволяющее выбрать источник данных (текст или файл), выполняемую операцию (зашифрование или расшифрование), просмотреть текущую ключевую пару, сгенерировать новую или задать её вручную. Для работы с большими целыми числами используется библиотека `num-bigint`, для проверки чисел на простоту – библиотека `num-prime`. Ключевая пара генерируется при запуске программы. Общий вид меню программы и сгенерированные ключи представлены на рисунке 2.1.

```
Generating keys...

Rabin cryptosystem - input/output: text
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): k

n : 98C29DDA98D3FF78BF773EE5138AAE004BC371A9502E40C22068F7C7F817C881FBF4CABC23B7AA59F3847CFA79394
7E7E97294B94775333D2B0B6281D1BDB558C522E80247212193C46696040A9EC18B21FBEA2CF68B7D5E13C8F81E2D94894A
6B18D9D60A4C540238F94ED5B5CD7E3C015AC2979180DC8E419D6F8D51A798DC6780F67CF2A343F928EF52A2F2B75268B82
A2A14A176D9FF5E91F489F5F7235B5E47E16856784F628A62EA04D1B84004218F191EC21032BF5FC749883DD08B71853073
443E1DBB579A6DE9EB1CA0ADC39F30191E224402D909F1FEFAEDF12D44F0C8CB668061004AEC314E1F44688B454B459FF1
91E55D95660A1578EEF6C06D6B57B0A788B9D6D5D45871CA7D2075D089F3AD76C8E6623D6D547A9D490523D2757672936BC
A9C08F75BDD99A42141C6D399EE1398C5DB1EBD2A0ED5080A3C32B3F87DBCC5D2A8DF966FCC462E789C361135BA5D31E298
00EA0F0E1FEC3E603F90E379F7A38089C3E311E9EC39EB169557A9918F4EF7054DCF958DD0D6657F5 (3072 bits)

p : CF481914D9C1C46B70E41051997AAC0ADF778A80FFA73A9068DA736BC23BB378C8A86EE3AD8FEFE04274BFD835FA
0AB6F6B6F638B9222E33D7AA15A78B34FC952682094776F0BC3FFDDBD3A07BA6916E8B9A4FEE2650C879EA7E0EDDC424F
9F54C29BE944BEBDF184CB4C18A33C28DE9FC7166BE8EB8BD244E730A63C998D83155BE30A4DCAEBC58F2C2A5D51D7A6
E4C6E10010BE450600CC17775661E17195AC10D656D0CAF88332DADCBS24C1A7CAE610011127C98B3D3DD4BA67EC7 (1536
bits)

q : C05E888E94DE5C4D5152C2255F140EA13F4E5A7AD63610F576E615A621363285EAFD1AC890BE9770F752C37E99034
8EB79F64EF310A35442AF4CAF39B4D47EAD96E890CC7729D391605ADEDB0E708E0DA53DA63D7BDB0E813E99782C426CF76A
9777E7319057FCFF801C28062FC5ACA6363759D32D13B064EFC2CEDF33332C192F2361B0C89DFE7E78EDAD09F8455319129
D60F625CBD63DE10BAFA5F2494DE2FB9F5AE2C9A17768EB898F4C36C042A8D492B1D703E5EE765C86382D4B52763 (1536
bits)
```

Рисунок 2.1 – Меню программы и ключевая пара

Закрытым ключом криптосистемы являются два различных простых числа p и q , удовлетворяющих условию $p \equiv q \equiv 3 \pmod{4}$, открытым – их произведение $n = pq$. Каждое из простых чисел имеет длину 1536 бит, поэтому длина модуля n составляет 3072 бита, или $k = 384$ байта. При генерации случайного числа устанавливаются старший бит, что гарантирует требуемую длину, и два младших бита, что обеспечивает остаток 3 при делении на 4; кандидаты перебираются до тех пор, пока не будет найдено простое число. Вместе с p и q в закрытом ключе сохраняются обратные элементы $y_p = p^{q-2} \bmod q$ и $y_q = q^{p-2} \bmod p$, вычисленные по малой теореме Ферма, чтобы не вычислять их повторно для каждого блока.

Зашифрование числа $m < n$ выполняется возведением в квадрат по модулю n : $c = m^2 \bmod n$. Расшифрование сводится к извлечению квадратного корня. Для простого $p \equiv 3 \pmod{4}$ корень по модулю p вычисляется явно как $m_p =$

$c^{\frac{p+1}{4}} \bmod p$, аналогично находится m_q , после чего корни по модулю n восстанавливаются по китайской теореме об остатках:

$$r_{1,2} = \pm(y_p p m_q + y_q q m_p) \bmod n, \quad r_{3,4} = \pm(y_p p m_q - y_q q m_p) \bmod n. \quad (2.1)$$

Первое слагаемое сравнимо с нулём по модулю p и с m_q по модулю q , второе – наоборот, поэтому четыре сочетания знаков дают все четыре корня. Извлечь корень, не зная разложения n , так же трудно, как разложить n на множители, на чём и основана стойкость криптосистемы.

Поскольку у каждого шифротекста четыре прообраза, открытый текст должен содержать избыточность, позволяющую выбрать среди корней истинный. Сообщение разбивается на куски по $D = k - R - 2 = 366$ байт, где $R = 16$ – длина избыточности. Каждый кусок дополняется копией своих последних R байт и маркерным байтом 01, занимающим старшую позицию, и полученная строка из $k - 1 = 383$ байт интерпретируется как число m . Такая длина гарантирует $m < n$, а ненулевой старший байт – $m^2 > n$, без чего возведение в квадрат не затрагивало бы модуль и шифротекст раскрывался бы извлечением обычного целого корня. Шифроблок записывается ровно в k байт. При расшифровании истинным признаётся корень длиной $k - 1$ байт с маркером на старшей позиции и совпадающей избыточностью; вероятность того, что проверку случайно пройдёт ложный корень, не превышает $3 \cdot 2^{-128}$. Блок-схема алгоритма приведена в приложении Б.

Перед разбиением на блоки сообщение дополняется байтом 80 и нулями до длины, кратной D , причём дополнение добавляется всегда, в том числе к сообщению, длина которого уже кратна D . Благодаря этому при расшифровании дополнение снимается однозначно: отбрасываются завершающие нули и байт 80 перед ними. Длина шифротекста всегда кратна 384 байтам: сообщение длиной до 365 байт занимает один блок.

Перед расшифрованием проверяется, что длина шифротекста ненулевая и кратна длине блока. Если ни один из четырёх корней какого-либо блока не проходит проверку избыточности либо дополнение после расшифрования оказывается некорректным, программа сообщает об искажении шифротекста.

Корректность реализации проверена зашифрованием и последующим расшифрованием сообщений длиной от 0 до 5000 байт, включая граничные длины 365, 366 и 367 байт, текстов на кириллице и файлов: во всех случаях исходное сообщение восстанавливается без искажений, а изменение одного бита шифротекста обнаруживается.

Исходный код разработанного программного средства приведён в приложении А: реализация криптосистемы представлена в листинге А.1, точка входа в программу – в листинге А.2.

2.1 Зашифрование и расшифрование текста

Результат работы программы для текста, вводимого с клавиатуры, представлен на рисунке 2.2. Сообщение длиной 25 байт укладывается в один блок, поэтому шифротекст занимает 384 байта. Криптосистема детерминирована: повторное зашифрование того же сообщения тем же ключом даёт тот же шифротекст.

```
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 1

Plaintext: Привет, Рабин!
mode      : encryption
plaintext : 25 bytes
encrypted : B0F595322A9A2FFC237ACC828C69EC2B046253A37D42A5B9713C7497E58C7398928B0575845CFD70CD234
9B14B7780B200E4EF3C293ED9143149B79A83349C2B8A58D1C8BEFD92103A64B1F84555C5428A803978A76C300D6CB55CCD
7AFC5625CC5D57FDDCA038DB2ED94A94293A4A335E71710D5BF4097DA6984B8A75A190CF41542B8004922D478182C805F3F
71A05AC20867F1B5AC1E7ACB71E90E7F30DAD0DF99FF6C5408293EB1A48A67B4F3F534EB62B9421649BF1B924501F8B8C47
696B99C1787FF8006DE81ED0E3B201ADBBA7C8BDE60EA5BFD81606594B6D107F5D645BD5B9A648A59A745984230AFC8AE61
3B4C832B8E1B0E672921454484479FCDA2A71E898FAD060EA86E76E2694ACD2DC7F62095C18EC3F06EEF5FD759140880B2
4D121A57EFB177209C2D75DDF405F09D6DF4B73D20FA1D7201319E9F07C795AF653AE4A66B60FD1B0E9E39296547999081B
83BF063E8BF8222AF9F40420EC6E56DCEEE897F3AAA7A73662A8DA229A1D4B43C6687A954F7EAF9AD2E998C34 (384 byte
s)

Rabin cryptosystem - input/output: text
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 2

Ciphertext (hex): B0F595322A9A2FFC237ACC828C69EC2B046253A37D42A5B9713C7497E58C7398928B0575845CFD70C
D2349B14B7780B200E4EF3C293ED9143149B79A83349C2B8A58D1C8BEFD92103A64B1F84555C5428A803978A76C300D6CB5
5CCD7AFC5625CC5D57FDDCA038DB2ED94A94293A4A335E71710D5BF4097DA6984B8A75A190CF41542B8004922D478182C80
5F3F71A05AC20867F1B5AC1E7ACB71E90E7F30DAD0DF99FF6C5408293EB1A48A67B4F3F534EB62B9421649BF1B924501F8B
8C47696B99C1787FF8006DE81ED0E3B201ADBBA7C8BDE60EA5BFD81606594B6D107F5D645BD5B9A648A59A745984230AFC8
AE613B4C832B8E1B0E672921454484479FCDA2A71E898FAD060EA86E76E2694ACD2DC7F62095C18EC3F06EEF5FD7591408
80B24D121A57EFB177209C2D75DDF405F09D6DF4B73D20FA1D7201319E9F07C795AF653AE4A66B60FD1B0E9E39296547999
081B83BF063E8BF8222AF9F40420EC6E56DCEEE897F3AAA7A73662A8DA229A1D4B43C6687A954F7EAF9AD2E998C34
mode      : decryption
encrypted : 384 bytes
decrypted : Привет, Рабин! (25 bytes)
```

Рисунок 2.2 – Зашифрование и расшифрование текста

2.2 Работа с файлами

Результат зашифрования и расшифрования файла представлен на рисунке 2.3. По умолчанию к имени зашифрованного файла добавляется расширение .rabin, а при расшифровании оно отбрасывается; если файл с таким именем уже существует, программа запрашивает подтверждение перезаписи.

```

Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): m

Switched to file mode

Rabin cryptosystem - input/output: file
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 1

Plaintext file: message.txt
Output file [message.txt.rabin]:
mode      : encryption
input     : message.txt (269 bytes)
output    : message.txt.rabin (384 bytes)

Rabin cryptosystem - input/output: file
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 2

Ciphertext file: message.txt.rabin
Output file [message.txt]: restored.txt
mode      : decryption
input     : message.txt.rabin (384 bytes)
output    : restored.txt (269 bytes)

```

Рисунок 2.3 – Зашифрование и расшифрование файла

2.3 Обнаружение искажений

Изменение шифротекста обнаруживается при расшифровании, как показано на рисунке 2.4. После изменения одной шестнадцатеричной цифры ни один из четырёх корней блока не содержит корректной избыточности, а шифротекст, укороченный на один байт, отвергается ещё до расшифрования, поскольку его длина не кратна длине блока.

```

Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 2

Ciphertext (hex): B0F595322A0A2FFC237ACC828C69EC2B046253A37D42A5B9713C7497E58C7398928B0575845CFD70C
D2349B14B7780B200E4EF3C293ED9143149B79A83349C2B8A58D1C8BEFD92103A64B1F84555C5428A803978A76C300D6CB5
5CCD7AFC5625CC5D57FDDCA038DB2ED94A94293A4A335E71710D5BF4097DA6984B8A75A190CF41542B8004922D478182C80
5F3F71A05AC20867F1B5AC1E7ACB71E90E7F30DAD0DF99FF6C5408293EB1A48A67B4F3F534EB62B9421649BF1B924501F8B
8C47696B99C1787FF8006DE81ED0E3B201ADBBA7C8BDE60EA5BFD81606594B6D107F5D645BD5B9A648A59A745984230AFC8
AE613B4C832B8E1B0E672921454484479FCDDA2A71E898FAD060EA86E76E2694ACD2DC7F62095C18EC3F06EEF5FD7591408
80B24D121A57EFB177209C2D75DDF405F09D6DF4B73D20FA1D7201319E9F07C795AF653AE4A66B60FD1B0E9E39296547999
081B83BF063E8BF8222AF9F40420EC6E56DCEEE897F3AAA7A73662A8DA229A1D4B43C6687A954F7EAF9D2E998C34
Error: Block corrupted

Rabin cryptosystem - input/output: text
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): 2

Ciphertext (hex): B0F595322A9A2FFC237ACC828C69EC2B046253A37D42A5B9713C7497E58C7398928B0575845CFD70C
D2349B14B7780B200E4EF3C293ED9143149B79A83349C2B8A58D1C8BEFD92103A64B1F84555C5428A803978A76C300D6CB5
5CCD7AFC5625CC5D57FDDCA038DB2ED94A94293A4A335E71710D5BF4097DA6984B8A75A190CF41542B8004922D478182C80
5F3F71A05AC20867F1B5AC1E7ACB71E90E7F30DAD0DF99FF6C5408293EB1A48A67B4F3F534EB62B9421649BF1B924501F8B
8C47696B99C1787FF8006DE81ED0E3B201ADBBA7C8BDE60EA5BFD81606594B6D107F5D645BD5B9A648A59A745984230AFC8
AE613B4C832B8E1B0E672921454484479FCDDA2A71E898FAD060EA86E76E2694ACD2DC7F62095C18EC3F06EEF5FD7591408
80B24D121A57EFB177209C2D75DDF405F09D6DF4B73D20FA1D7201319E9F07C795AF653AE4A66B60FD1B0E9E39296547999
081B83BF063E8BF8222AF9F40420EC6E56DCEEE897F3AAA7A73662A8DA229A1D4B43C6687A954F7EAF9D2E998C
Error: Invalid message length

```

Рисунок 2.4 – Расшифрование искажённого шифротекста

2.4 Задание ключей

Помимо генерации ключевой пары программа позволяет задать простые числа p и q вручную в шестнадцатеричной записи. Введённые значения проверяются: оба числа должны быть простыми, сравнимыми с 3 по модулю 4 и различными, а их произведение – иметь длину не менее $2R + 2 = 34$ байт, чтобы в блоке поместились данные, избыточность и маркер. Результат ввода некорректных ключей и генерации новой пары представлен на рисунке 2.5.

```
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): m

Switched to text mode

Rabin cryptosystem - input/output: text
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): c

p (hex): 1D
Error: 1D is not 3 mod 4

Rabin cryptosystem - input/output: text
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): c

p (hex): 1F
q (hex): 23
Error: 23 is not prime

Rabin cryptosystem - input/output: text
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'c' to change keys,
'q' to quit): g

Generating keys...
```

Рисунок 2.5 – Проверка вводимых ключей и генерация новой пары

Таким образом криптосистема Рабина была изучена, реализована и протестирована в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения асимметричных криптосистем на примере криптосистемы Рабина. Было разработано программное средство на языке *Rust*, реализующее генерацию ключевой пары с 3072-битным модулем, зашифрование открытым ключом и расшифрование закрытым ключом над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование восстанавливает исходное сообщение без искажений при любой длине, а искажение шифротекста обнаруживается благодаря избыточности, добавляемой к каждому блоку. Работа показала основную особенность криптосистемы: функция зашифрования не является взаимно однозначной, и у каждого шифротекста есть четыре прообраза, поэтому для однозначного расшифрования в открытый текст приходится вносить избыточность. Кроме того, зашифрование требует всего одного умножения по модулю и выполняется значительно быстрее расшифрования, а стойкость криптосистемы доказуемо сводится к задаче разложения модуля на множители.

Отдельно следует отметить отличия от изученных ранее симметричных шифров ГОСТ 28147-89 и СТБ 34.101.31-2011: ключ зашифрования может быть открыт, так как знание n не позволяет расшифровывать без знания p и q ; длина блока определяется длиной модуля и составляет сотни байт; шифротекст длиннее открытого текста на величину избыточности и маркера в каждом блоке, а преобразования выполняются над большими целыми числами, а не над машинными словами.

Все задачи лабораторной работы выполнены в полном объеме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Реализация криптосистемы Рабина на *Rust*

```
use num_bigint::{BigUint, RandBigInt};
use num_prime::nt_funcs::is_prime;
use rand::rngs::OsRng;
use rand::{SeedableRng, rngs::StdRng};

pub const REDUNDANCY_SIZE: usize = 16;

pub struct PublicKey {
    n: BigUint,
}

impl PublicKey {
    pub fn new(n: BigUint) -> Self {
        return Self { n };
    }

    pub fn n(&self) -> &BigUint {
        return &self.n;
    }
}

pub struct PrivateKey {
    public: PublicKey,
    p: BigUint,
    q: BigUint,
    yp: BigUint,
    yq: BigUint,
}

impl PrivateKey {
    pub fn new(p: BigUint, q: BigUint) -> Self {
        let big_two = BigUint::from(2u32);

        let yp = p.modpow(&(&q - &big_two), &q);
        let yq = q.modpow(&(&p - &big_two), &p);
        return Self {
            public: PublicKey { n: &p * &q },
            p,
            q,
            yp,
            yq,
        };
    }

    pub fn p(&self) -> &BigUint {
        return &self.p;
    }

    pub fn q(&self) -> &BigUint {
        return &self.q;
    }
}

pub struct Rabin {}

impl Rabin {
```

```

fn gen_prime_3mod4(bits: u64) -> BigUint {
    let mut rng = StdRng::from_rng(&mut OsRng).unwrap();
    loop {
        let mut candidate = rng.gen_biguint(bits);
        candidate.set_bit(bits - 1, true);
        candidate.set_bit(1, true);
        candidate.set_bit(0, true);

        if is_prime(&candidate, None).probably() {
            return candidate;
        }
    }
}

fn get_k_from_key(key: &PublicKey) -> usize {
    return key.n.bits().div_ceil(8) as usize;
}

pub fn keygen() -> (PrivateKey, PublicKey) {
    let p = Self::gen_prime_3mod4(1536);
    let mut q = Self::gen_prime_3mod4(1536);
    while q == p {
        q = Self::gen_prime_3mod4(1536);
    }
    let n = &p * &q;

    return (PrivateKey::new(p, q), PublicKey::new(n));
}

fn encrypt_block(block: &[u8], key: &PublicKey) -> Vec<u8> {
    let block_size: usize = Self::get_k_from_key(key) - 1;
    let mut m_bytes = vec![0u8; block_size];
    m_bytes[..block.len()].copy_from_slice(block);
    m_bytes.copy_within((block.len() - REDUNDANCY_SIZE)..block.len(),
block.len());
    m_bytes[block_size - 1] = 0x01;

    let m = BigUint::from_bytes_le(&m_bytes);
    let c = m.modpow(&BigUint::from(2u32), &key.n);

    let mut res = c.to_bytes_le();
    res.resize(block_size + 1, 0);

    return res;
}

fn decrypt_block(block: &[u8], key: &PrivateKey) -> Result<Vec<u8>,
String> {
    let block_size: usize = Self::get_k_from_key(&key.public);
    let mut c_bytes = vec![0u8; block_size];
    c_bytes.copy_from_slice(block);

    let mut results: Vec<BigUint> = Vec::with_capacity(4);

    let big_one = BigUint::from(1u32);
    let big_four = BigUint::from(4u32);
    let c = BigUint::from_bytes_le(&c_bytes);
    let p = &key.p;
    let q = &key.q;
    let n = &key.public.n;

    let mp = c.modpow(&((p + &big_one) / &big_four), p);
    let mq = c.modpow(&((q + &big_one) / &big_four), q);

```

```

let yp = &key.yp;
let yq = &key.yq;

let t1 = yp * p * &mq;
let t2 = yq * q * &mp;

results.push((&t1 + &t2) % n);
results.push(n - &results[0]);
results.push(((&t1 % n) + n - (&t2 % n)) % n);
results.push(n - &results[2]);

for result in results {
    let redundancy_start = block_size - 2 - REDUNDANCY_SIZE;
    let possible_copy_start = redundancy_start - REDUNDANCY_SIZE;
    let mut res_bytes = result.to_bytes_le();
    if res_bytes.len() != block_size - 1 {
        continue;
    } else if res_bytes[block_size - 2] != 0x01 {
        continue;
    } else if res_bytes[possible_copy_start..redundancy_start]
        == res_bytes[redundancy_start..(block_size - 2)]
    {
        res_bytes.truncate(redundancy_start);
        return Ok(res_bytes);
    }
}

return Err("Block corrupted".to_owned());
}

pub fn encrypt(mut plain_bytes: Vec<u8>, key: &PublicKey) -> Vec<u8> {
    let block_size: usize = Self::get_k_from_key(key) - 1;
    let payload_size = block_size - REDUNDANCY_SIZE as usize - 1;

    let mut result: Vec<u8> = vec![];

    let padding = payload_size - plain_bytes.len() % payload_size;
    plain_bytes.push(0x80);
    for _ in 0..(padding - 1) {
        plain_bytes.push(0);
    }

    let blocks = plain_bytes.chunks(payload_size);
    for block in blocks {
        result.extend_from_slice(&Self::encrypt_block(block, key));
    }

    return result;
}

pub fn decrypt(encrypted_bytes: Vec<u8>, key: &PrivateKey) ->
Result<Vec<u8>, String> {
    let block_size = Self::get_k_from_key(&key.public);
    if encrypted_bytes.is_empty() || encrypted_bytes.len() % block_size != 0 {
        return Err("Invalid message length".to_owned());
    }

    let mut result: Vec<u8> = vec![];

    let blocks = encrypted_bytes.chunks(block_size);
    for block in blocks {

```

```

        let decrypted = Self::decrypt_block(block, key)?;
        result.extend_from_slice(&decrypted);
    }

    let payload_size = block_size - 2 - REDUNDANCY_SIZE;
    match result.iter().rposition(|&b| b != 0) {
        Some(pos) if result[pos] == 0x80 && pos >= result.len() -
payload_size => {
            result.truncate(pos);
        }
        _ => return Err("Invalid padding".to_owned()),
    }

    return Ok(result);
}
}

```

Листинг А.2 – Точка входа в программу

```

mod encryption;
use common::cli::{self, IoMode};
use encryption::rabin::*;
use num_bigint::BigUint;
use num_prime::nt_funcs::is_prime;

const TITLE: &str = "Rabin cryptosystem";
const ENCRYPTED_EXTENSION: &str = "rabin";
/// A block holds at least as many data bytes as the redundancy copies from
it,
/// plus the redundancy itself and the marker, and the modulus is one byte
wider.
const MIN_MODULUS_BYTES: usize = 2 * REDUNDANCY_SIZE + 2;

fn generate_keys() -> (PrivateKey, PublicKey) {
    println!("Generating keys...");

    return Rabin::keygen();
}

fn parse_prime(label: &str) -> Result<BigUint, String> {
    let digits: String = cli::prompt(label)
        .chars()
        .filter(|c| !c.is_whitespace())
        .collect();
    let prime = BigUint::parse_bytes(digits.as_bytes(), 16)
        .ok_or_else(|| format!("{label:?} expects a hex number"))?;

    if &prime % 4u32 != BigUint::from(3u32) {
        return Err(format!("{prime:X} is not 3 mod 4"));
    }
    if !is_prime(&prime, None).probably() {
        return Err(format!("{prime:X} is not prime"));
    }

    return Ok(prime);
}

fn read_keys() -> Result<(PrivateKey, PublicKey), String> {
    let p = parse_prime("p (hex): ")?;
    let q = parse_prime("q (hex): ")?;
    if p == q {
        return Err("p and q must differ".to_string());
    }
}

```

```

    let n = &p * &q;
    if (n.bits().div_ceil(8) as usize) < MIN_MODULUS_BYTES {
        return Err(format!(
            "n = p·q must be at least {MIN_MODULUS_BYTES} bytes long"
        ));
    }

    return Ok((PrivateKey::new(p, q), PublicKey::new(n)));
}

fn print_keys(private: &PrivateKey, public: &PublicKey) {
    println!("  n : {:X} ({} bits)", public.n(), public.n().bits());
    println!("  p : {:X} ({} bits)", private.p(), private.p().bits());
    println!("  q : {:X} ({} bits)", private.q(), private.q().bits());
}

fn run_cipher(
    io_mode: IoMode,
    encrypt: bool,
    private: &PrivateKey,
    public: &PublicKey,
) -> Result<(), String> {
    let (input, source) = cli::read_payload(io_mode, encrypt)?;
    if input.is_empty() {
        return Err("nothing to process: the input is empty".to_string());
    }

    let destination = match &source {
        Some(path) => Some(cli::ask_output_path(&cli::default_output_path(
            path,
            ENCRYPTED_EXTENSION,
            encrypt,
        ))?),
        None => None,
    };

    let output = if encrypt {
        Rabin::encrypt(input.clone(), public)
    } else {
        Rabin::decrypt(input.clone(), private)?
    };

    println!(
        "  mode      : {}",
        if encrypt { "encryption" } else { "decryption" }
    );

    return cli::emit_output(
        encrypt,
        &input,
        &output,
        source.as_deref(),
        destination.as_deref(),
    );
}

fn main() {
    let (mut private, mut public) = generate_keys();
    let mut io_mode = IoMode::Text;

    loop {
        println!();

```

```

println!("{TITLE} - input/output: {}", io_mode.name());
println!("{:<22}2) decrypt", "1) encrypt");

let choice = cli::prompt(
    "Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to
generate keys, 'c' to change keys, 'q' to quit): ",
);
println!();

let result = match choice.as_str() {
    "1" => run_cipher(io_mode, true, &private, &public),
    "2" => run_cipher(io_mode, false, &private, &public),
    "m" => {
        io_mode = io_mode.toggled();
        println!("Switched to {} mode", io_mode.name());
        Ok(())
    }
    "k" => {
        print_keys(&private, &public);
        Ok(())
    }
    "g" => {
        (private, public) = generate_keys();
        Ok(())
    }
    "c" => read_keys().map(|keys| (private, public) = keys),
    "q" => break,
    _ => Err(format!("unknown choice: {choice:?}")),
};

if let Err(err) = result {
    println!("Error: {err}");
}
}
}

```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схема алгоритма**

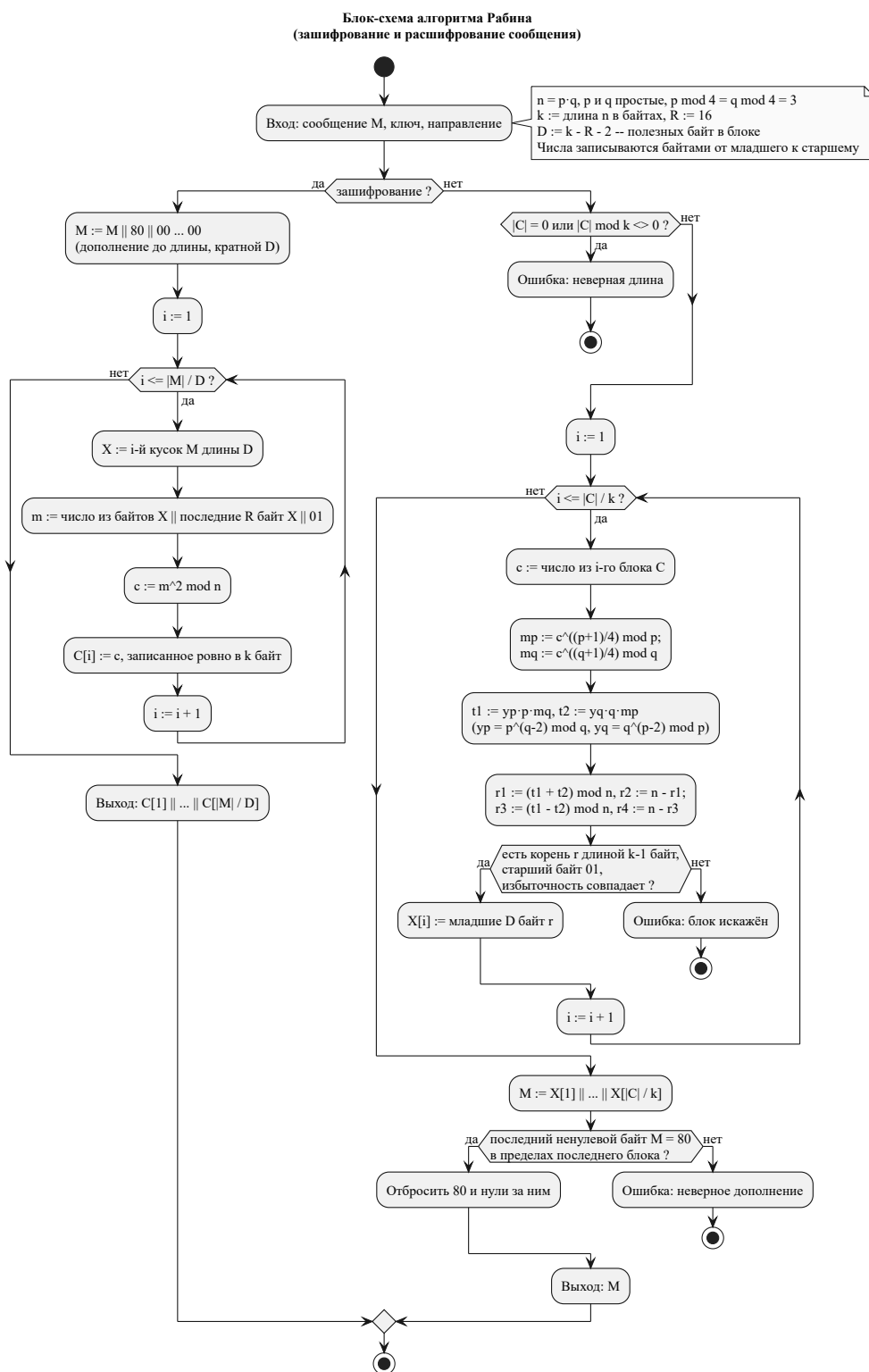


Рисунок Б.1 – Блок-схема алгоритма